

IES POLITÉCNICO HERMENEGILDO LANZ

Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Multiplataforma — 2º Curso

PROYECTO INTERMODULAR

Sistema de Gestión de Taller Mecánico

Alumno: Samuel Pozo Olvera

Tutor: IES Politécnico Hermenegildo Lanz

Curso académico: 2025 / 2026

ÍNDICE

1. Análisis del Problema	3
1.1. Introducción	
1.2. Objetivos Principales	
1.3. Funciones y Rendimientos Deseados	
1.4. Planteamiento y Evaluación de Diversas Soluciones	
1.5. Justificación de la Solución Elegida	
1.6. Modelado de la Solución	
1.7. Planificación Temporal	
2. Diseño e Implementación del Proyecto	8
2.1. Diseño de la Base de Datos	
2.2. Arquitectura del Sistema	
2.3. Backend — API REST con Spring Boot	
2.4. Aplicación de Escritorio — JavaFX	
2.5. Aplicación Móvil — React Native	
2.6. Diseño Estético	
3. Fase de Pruebas	38
4. Documentación de la Aplicación	42
5. Conclusiones Finales	48
6. Bibliografía	51

1. ANÁLISIS DEL PROBLEMA

1.1. Introducción

El presente proyecto consiste en el desarrollo de un sistema integral de gestión para un taller mecánico, compuesto por tres módulos software interconectados: una aplicación de escritorio destinada al administrador del taller, una aplicación móvil con sesión diferenciada para mecánicos y clientes, y una API REST como capa de comunicación común entre todas las plataformas.

La gestión de un taller mecánico implica una gran cantidad de procesos: la recepción de vehículos, la asignación de trabajos a mecánicos, el seguimiento del estado de cada reparación, la comunicación con los clientes, la gestión de horarios y ausencias del personal, y el control del espacio físico del taller. Actualmente muchos talleres de pequeño y mediano tamaño gestionan estos procesos de forma manual o con herramientas no especializadas, lo que genera ineficiencias, falta de trazabilidad y una comunicación deficiente con el cliente.

Este proyecto nace con la intención de digitalizar y centralizar todos estos procesos en un único sistema, accesible desde múltiples dispositivos y plataformas, que proporcione a cada tipo de usuario las herramientas necesarias para realizar su trabajo de forma eficiente y transparente.

1.2. Objetivos Principales

- Desarrollar una aplicación de escritorio multiplataforma con control total sobre todos los datos del sistema, destinada al administrador del taller.
- Desarrollar una aplicación móvil con roles diferenciados para mecánicos y clientes, adaptando la interfaz y las funcionalidades a las necesidades de cada perfil.
- Implementar un sistema de notificaciones automáticas que mantenga informados a todos los usuarios sobre los cambios relevantes en el sistema.
- Gestionar el estado de los vehículos en el taller y proporcionar al cliente información en tiempo real sobre el estado de su reparación.
- Diseñar e implementar una API REST robusta, segura y bien estructurada como núcleo de comunicación entre todas las plataformas.
- Garantizar la seguridad del sistema mediante autenticación JWT y control de acceso por roles.
- Implementar validaciones lógicas en todos los formularios para asegurar la integridad de los datos (email, teléfono, NIF, matrícula).
- Proporcionar una experiencia de usuario coherente y estéticamente cuidada en todas las plataformas.

1.3. Funciones y Rendimientos Deseados

1.3.1. Aplicación de Escritorio (Administrador)

La aplicación de escritorio está destinada exclusivamente al administrador del taller, que actuará como superusuario con control total sobre el sistema:

- Acceso seguro mediante autenticación JWT con visualización opcional de la contraseña.
- Dashboard con resumen en tiempo real: vehículos en taller, órdenes pendientes, trabajos en proceso, vehículos listos para entregar, mapa visual de plazas y panel de notificaciones recientes.

- Gestión completa (CRUD) de mecánicos: alta con generación de credenciales de acceso, edición y baja.
- Gestión completa de clientes: alta manual o registro automático desde la app móvil, edición y baja.
- Gestión completa de vehículos con asignación al cliente propietario.
- Gestión completa de órdenes de trabajo: creación, seguimiento, cambio de estado con observación, visualización del historial completo y eliminación.
- Gestión de horarios de mecánicos con notificación automática al crear o eliminar turnos.
- Gestión de solicitudes de ausencia: aprobación o rechazo con notificación automática al mecánico.
- Gestión del espacio físico: plazas de estacionamiento con estado visual en tiempo real.
- Validaciones en todos los formularios con mensajes de error descriptivos sin cerrar el formulario.

1.3.2. Aplicación Móvil — Rol Mecánico

- Acceso con las credenciales proporcionadas por el administrador.
- Visualización de las órdenes de trabajo asignadas con detalle completo e historial de cambios.
- Actualización del estado y diagnóstico de cada orden directamente desde el móvil.
- Consulta del horario semanal asignado.
- Solicitud de ausencias y días libres con validación de fechas.
- Centro de notificaciones con badge de mensajes no leídos.
- Perfil con foto seleccionable desde la galería del dispositivo.

1.3.3. Aplicación Móvil — Rol Cliente

- Registro propio desde la aplicación o mediante credenciales del administrador.
- Consulta de vehículos registrados a su nombre.
- Seguimiento en tiempo real del estado de sus reparaciones con historial completo de cambios y diagnósticos.
- Centro de notificaciones sobre novedades en sus vehículos.
- Perfil con foto seleccionable desde la galería del dispositivo.

1.4. Planteamiento y Evaluación de Diversas Soluciones

1.4.1. Alternativas para la Aplicación de Escritorio

Tecnología	Ventajas	Inconvenientes	Decisión
JavaFX	Integración nativa con Java, interfaz rica personalizable, buen rendimiento	Curva de aprendizaje en FXML y CSS	ELEGIDA
Electron	Multiplataforma, HTML/CSS/JS familiar	Alto consumo de memoria, menor rendimiento nativo	Descartada
Qt	Muy potente y multiplataforma	Requiere C++, fuera del stack del ciclo	Descartada

1.4.2. Alternativas para la Aplicación Móvil

Tecnología	Ventajas	Inconvenientes	Decisión
React Native + TypeScript	Multiplataforma, tipado estático, gran ecosistema, Expo	Rendimiento ligeramente inferior al nativo	ELEGIDA
Flutter	Buen rendimiento, UI consistente	Requiere aprender Dart	Descartada
Nativo Android/iOS	Máximo rendimiento	Duplica el esfuerzo de desarrollo	Descartada

1.4.3. Alternativas para el Backend

Tecnología	Ventajas	Inconvenientes	Decisión
Spring Boot	Robusto, Spring Security, JPA, ecosistema maduro, Java	Verboso, mayor configuración inicial	ELEGIDA
Node.js + Express	Ligero, rápido de montar	Menos estructurado, sin tipado estático nativo	Descartada
Django REST	Potente, admin panel incluido	Introduce Python como lenguaje adicional	Descartada

1.5. Justificación de la Solución Elegida

El stack tecnológico seleccionado aprovecha los conocimientos adquiridos durante el ciclo formativo y garantiza una integración coherente entre todos los módulos del sistema:

- Backend: Spring Boot 3.5 con Java 24, Spring Security y JWT. Proporciona una API REST segura, escalable y bien documentada.
- Base de datos: MySQL con JPA/Hibernate. Fiabilidad probada, compatibilidad total con Spring Boot y gestión sencilla mediante phpMyAdmin.
- App de escritorio: JavaFX 21 con Java 21. Permite crear interfaces ricas y personalizadas con tema oscuro propio, integrándose perfectamente con el ecosistema Java del backend.
- App móvil: React Native con Expo SDK 54 y TypeScript. Desarrollo multiplataforma (Android/iOS) con tipado estático que reduce errores y mejora el mantenimiento.

1.6. Modelado de la Solución

1.6.1. Recursos Humanos

El proyecto es desarrollado íntegramente de forma individual por un único desarrollador. Este, asume todos los roles del proyecto: analista de requisitos, arquitecto del sistema, diseñador de base de datos, desarrollador backend, desarrollador frontend de escritorio, desarrollador móvil y documentador.

1.6.2. Recursos Hardware

- Ordenador de desarrollo con sistema operativo Windows 10/11.
- Emulador Android Pixel 6 con API 34 (Android 14) gestionado mediante Android Studio para las pruebas de la aplicación móvil.
- Servidor local XAMPP con MySQL para la base de datos durante el desarrollo.

1.6.3. Recursos Software

Herramienta	Versión	Uso
IntelliJ IDEA	2024.x	Desarrollo del backend Spring Boot y la app JavaFX
Visual Studio Code	1.9x	Desarrollo de la app móvil React Native
Android Studio	2024.x	Gestión del emulador Android
XAMPP	8.2	Servidor local con MySQL y phpMyAdmin
Postman	11.x	Pruebas y documentación de la API REST
Git / GitHub	2.x	Control de versiones del proyecto
Node.js	18.x	Runtime para React Native y Expo
Java JDK	21 / 24	JDK 21 para JavaFX, JDK 24 para Spring Boot
Maven	3.x	Gestión de dependencias del backend y escritorio
npm	10.x	Gestión de dependencias de la app móvil

1.7. Planificación Temporal

El proyecto se ha desarrollado a lo largo del tercer trimestre del curso 2025/2026, con seis entregas de seguimiento y una entrega final:

Entrega	Fecha	Contenido
Vídeo 1	13/03/2026	Diseño de la base de datos, diagrama E/R y script SQL completo.
Vídeo 2	27/03/2026	Estructura del backend Spring Boot, entidades JPA, repositorios, seguridad JWT y endpoints de autenticación.
Vídeo 3	10/04/2026	Avance en la aplicación móvil (login, navegación por roles) y aplicación de escritorio (dashboard, órdenes, plazas).
Vídeo 4	24/04/2026	Sistema de notificaciones automáticas, funcionalidades avanzadas e integración entre plataformas.
Vídeo 5	08/05/2026	CRUD completo en escritorio, registro de clientes, solicitud de ausencias, detalle de órdenes e historial en móvil.
Vídeo 6	18/05/2026	Pulido de detalles, validaciones, foto de perfil, gestión de errores y documentación.
Entrega final	19/05/2026	Código completo, documentación y presentación PowerPoint.
Defensa	20/05/2026	Defensa oral del proyecto en el aula A13.

2. DISEÑO E IMPLEMENTACIÓN DEL PROYECTO

2.1. Diseño de la Base de Datos

La base de datos es el núcleo del sistema. Su diseño ha sido el punto de partida del proyecto, garantizando una estructura sólida, normalizada y escalable que soporte todas las funcionalidades del sistema. Se ha utilizado MySQL como SGBD y JPA/Hibernate para el mapeo objeto-relacional en el backend.

2.1.1. Diagrama Entidad-Relación

El sistema cuenta con 12 entidades que cubren todos los procesos del taller mecánico. Las relaciones entre entidades son las siguientes:

- USUARIO es la entidad base del sistema. Tanto MECANICO como CLIENTE tienen una relación 1:1 con USUARIO, extendiendo sus datos específicos del rol.
- VEHICULO pertenece a un CLIENTE (N:1) y puede tener múltiples ORDENES_TRABAJO.
- ORDEN_TRABAJO relaciona un VEHICULO con un MECANICO asignado. Genera HISTORIAL_ESTADO y NOTIFICACION.
- MECANICO tiene HORARIOS (1:N) y AUSENCIAS (1:N).
- PLAZA puede estar asociada a un ESTACIONAMIENTO activo.

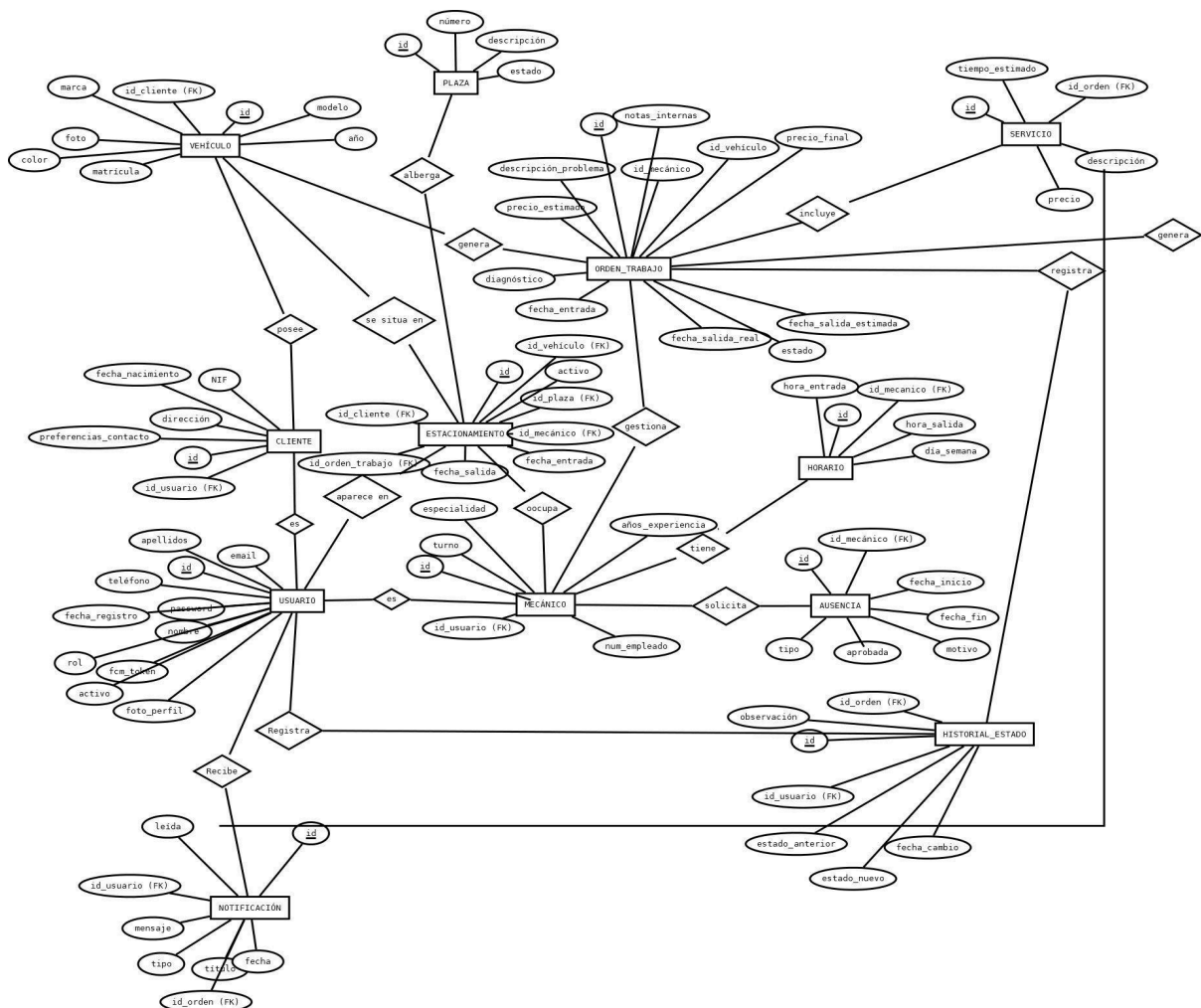


Figura 1: Diagrama Entidad-Relación del Sistema de Gestión de Taller Mecánico

2.1.2. Definición de Tablas

A continuación se presenta la definición completa de cada tabla con sus campos, tipos de datos y restricciones.

Tabla USUARIO — Entidad base para todos los usuarios del sistema

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
nombre	VARCHAR(100)	—	NOT NULL
apellidos	VARCHAR(150)	—	NOT NULL
email	VARCHAR(150)	—	NOT NULL, UNIQUE
password	VARCHAR(255)	—	NOT NULL, BCrypt hash
telefono	VARCHAR(15)	—	Opcional, formato español
foto_perfil	LONGTEXT	—	Imagen codificada en Base64
activo	BOOLEAN	—	DEFAULT TRUE
fecha_registro	DATETIME	—	DEFAULT CURRENT_TIMESTAMP
fcm_token	VARCHAR(255)	—	Token para notificaciones push (trabajo futuro)
rol	ENUM	—	ADMIN MECANICO CLIENTE

Tabla MECANICO — Datos específicos del personal técnico

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_usuario	INT	FK	→ USUARIO(id), UNIQUE
especialidad	VARCHAR(100)	—	Opcional (ej: Electricidad, Carrocería)
turno	ENUM	—	MANANA TARDE NOCHE
anos_experiencia	INT	—	DEFAULT 0
num_empleado	VARCHAR(20)	—	NOT NULL, UNIQUE

Tabla CLIENTE — Datos específicos de los clientes del taller

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_usuario	INT	FK	→ USUARIO(id), UNIQUE
nif	VARCHAR(9)	—	Formato español: 8 dígitos + letra validada
direccion	VARCHAR(255)	—	Opcional
fecha_nacimiento	DATE	—	Opcional
preferencias_contacto	VARCHAR(255)	—	Opcional

Tabla VEHICULO — Vehículos registrados en el sistema

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_cliente	INT	FK	→ CLIENTE(id)
matricula	VARCHAR(10)	—	NOT NULL, UNIQUE. Formato: 1234ABC
marca	VARCHAR(50)	—	NOT NULL
modelo	VARCHAR(50)	—	NOT NULL
anio	INT	—	Año de fabricación (1900-2026)
color	VARCHAR(30)	—	Opcional
foto	VARCHAR(255)	—	Opcional

Tabla ORDEN_TRABAJO — Núcleo operativo del sistema

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_vehiculo	INT	FK	→ VEHICULO(id)
id_mecanico	INT	FK	→ MECANICO(id)
estado	ENUM	—	PENDIENTE EN_PROCESO DIAGNOSTICADO REPARADO LISTO ENTREGADO
descripcion_problema	TEXT	—	NOT NULL. Mínimo 10 caracteres
diagnostico	TEXT	—	Actualizable por el mecánico
notas_internas	TEXT	—	Visibles solo para el admin y el mecánico
precio_estimado	DECIMAL(10,2)	—	Opcional
precio_final	DECIMAL(10,2)	—	Opcional
fecha_entrada	DATETIME	—	DEFAULT CURRENT_TIMESTAMP
fecha_salida_estimada	DATETIME	—	Opcional

Tabla HISTORIAL_ESTADO — Trazabilidad completa de cambios en órdenes

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_orden	INT	FK	→ ORDEN_TRABAJO(id)
id_usuario	INT	FK	→ USUARIO(id). Quien realizó el cambio
estado_anterior	VARCHAR(50)	—	NULL para el estado inicial. 'DIAGNOSTICO' para cambios de diagnóstico
estado_nuevo	VARCHAR(50)	—	NOT NULL
fecha_cambio	DATETIME	—	DEFAULT CURRENT_TIMESTAMP
observacion	TEXT	—	Comentario opcional sobre el cambio

Tabla NOTIFICACION — Sistema de avisos internos

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_usuario	INT	FK	→ USUARIO(id). Destinatario
id_orden	INT	FK	→ ORDEN_TRABAJO(id). Nullable
titulo	VARCHAR(100)	—	NOT NULL
mensaje	TEXT	—	NOT NULL
leida	BOOLEAN	—	DEFAULT FALSE
fecha	DATETIME	—	DEFAULT CURRENT_TIMESTAMP
tipo	ENUM	—	ESTADO CHAT AUSENCIA GENERAL

Tabla PLAZA — Espacios físicos del taller

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
numero	VARCHAR(10)	—	NOT NULL, UNIQUE
descripcion	VARCHAR(255)	—	Opcional
estado	ENUM	—	LIBRE OCUPADA RESERVADA MANTENIMIENTO

Tabla HORARIO — Turnos de trabajo de los mecánicos

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_mecanico	INT	FK	→ MECANICO(id)
dia_semana	ENUM	—	LUNES MARTES MIERCOLES JUEVES VIERNES SABADO DOMINGO
hora_entrada	TIME	—	NOT NULL. Formato HH:mm validado
hora_salida	TIME	—	NOT NULL. Debe ser posterior a hora_entrada

Tabla AUSENCIA — Solicitudes de ausencia del personal

Campo	Tipo	Clave	Restricciones
id	INT	PK	AUTO_INCREMENT
id_mecanico	INT	FK	→ MECANICO(id)
fecha_inicio	DATE	—	NOT NULL
fecha_fin	DATE	—	NOT NULL. Debe ser posterior o igual a fecha_inicio
tipo	ENUM	—	VACACIONES DIA_LIBRE BAJA OTRO
motivo	TEXT	—	Opcional

Campo	Tipo	Clave	Restricciones
aprobada	BOOLEAN	—	NULL=pendiente TRUE=aprobada FALSE=rechazada

2.2. Arquitectura del Sistema

El sistema sigue una arquitectura cliente-servidor en tres capas bien definidas. Esta arquitectura garantiza la separación de responsabilidades, facilita el mantenimiento y permite que nuevos clientes puedan integrarse en el futuro consumiendo la misma API REST.

- Capa de presentación: Aplicación de escritorio JavaFX y aplicación móvil React Native. Cada cliente gestiona su propia interfaz de usuario y se comunica con el backend mediante peticiones HTTP con JSON.
- Capa de negocio: API REST desarrollada con Spring Boot. Gestiona la lógica de negocio, la autenticación JWT, la autorización por roles, las validaciones y las notificaciones automáticas.
- Capa de datos: Base de datos MySQL gestionada mediante JPA/Hibernate. Toda la persistencia de datos pasa por esta capa.

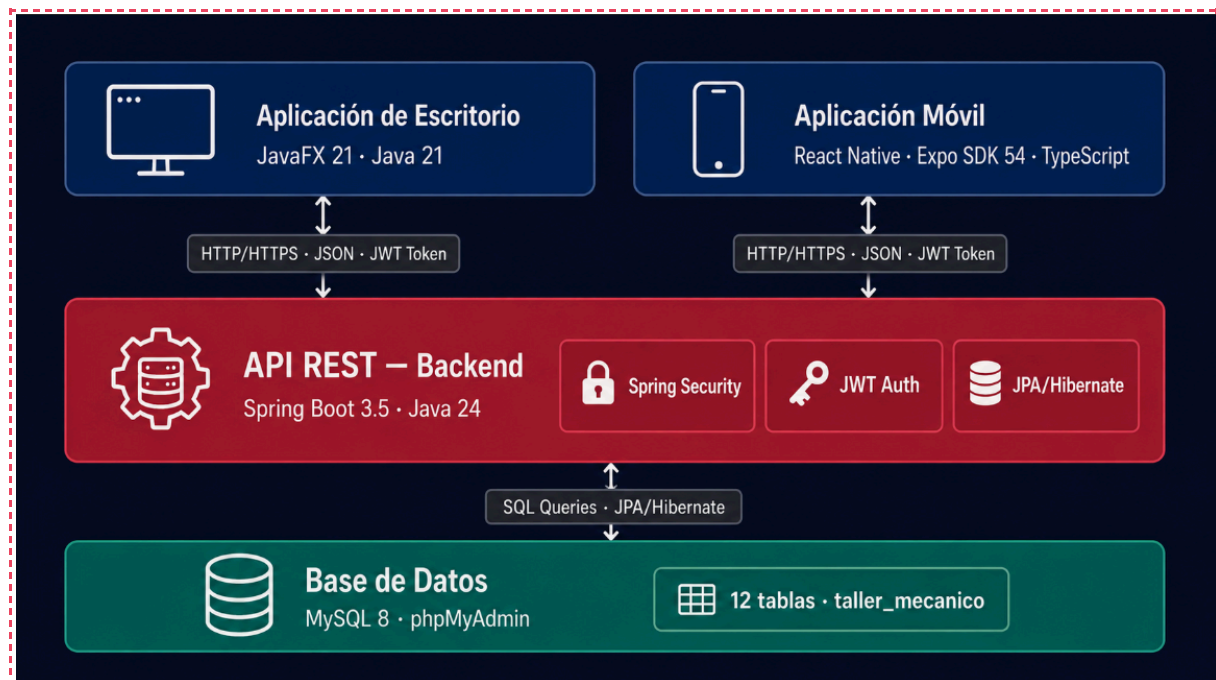


Figura 2: Arquitectura del sistema en tres capas

2.3. Backend — API REST con Spring Boot

2.3.1. Stack Tecnológico

Tecnología	Versión	Uso
Spring Boot	3.5	Framework principal del backend
Java	24.0.1	Lenguaje de programación
Spring Security	6.x	Autenticación y autorización por roles
JWT	0.12.6	Generación y validación de tokens JWT
Spring Data JPA	3.x	Mapeo objeto-relacional con Hibernate
MySQL Driver	8.x	Conector JDBC para MySQL
Validation	3.x	Validación de datos de entrada
Maven	3.x	Gestión de dependencias

2.3.2. Estructura del Proyecto

Paquete	Clases principales	Responsabilidad
model	Usuario, Mecanico, Cliente, Vehiculo, OrdenTrabajo, HistorialEstado, Notificacion, Plaza, Horario, Ausencia, Servicio, Estacionamiento	Entidades JPA. Sin Lombok por incompatibilidad con Java 24; getters/setters manuales.
repository	12 interfaces JpaRepository	Acceso a datos con métodos personalizados (findByEmail, findByMecanicold, etc.)
service	OrdenTrabajoService, AuthService, AusenciaScheduler	Lógica de negocio: creación de órdenes, cambio de estado, notificaciones automáticas y scheduler diario.
controller	AdminController, MecanicoController, ClienteController, AuthController, NotificacionController, SetupController	Exposición de endpoints REST con control de acceso por rol.
dto	LoginRequest, RegisterRequest, LoginResponse	Objetos de transferencia. LoginResponse es un Java Record con token, email, nombre, apellidos, rol, id, mecanicold y clienteld.
security	JwtAuthenticationFilter, JwtService, SecurityConfig, UserDetailsServiceImpl	Filtro JWT, generación/validación de tokens y configuración de Spring Security.

Paquete	Clases principales	Responsabilidad
exception	GlobalExceptionHandler, ResourceNotFoundException, BadRequestException	Gestión centralizada de errores con mensajes descriptivos para el usuario.

2.3.3. Seguridad JWT

El flujo de autenticación funciona de la siguiente manera:

1. El cliente envía email y contraseña al endpoint `/api/auth/login`.
2. El servidor valida las credenciales contra la base de datos (contraseña hasheada con BCrypt).
3. Si son correctas, genera un token JWT firmado con la clave secreta, con expiración de 24 horas.
4. El token incluye el email del usuario como subject y se devuelve junto con el rol, nombre, id, mecanicold y clienteld.
5. En cada petición posterior, el cliente incluye el token en la cabecera `Authorization: Bearer {token}`.
6. El filtro `JwtAuthenticationFilter` intercepta cada petición, extrae y valida el token, y establece el contexto de seguridad con el usuario y sus roles.

Las rutas públicas (sin autenticación requerida) son:

- `/api/auth/login` y `/api/auth/register` — Autenticación y registro.
- `/api/admin/setup` — Creación del primer administrador (se bloquea automáticamente si ya existe un admin).

2.3.4. Endpoints Implementados

Módulo de Autenticación (`/api/auth`)

Método	Ruta	Acceso	Descripción
POST	<code>/api/auth/register</code>	Público	Registra un nuevo usuario con rol CLIENTE. Devuelve token JWT y datos del usuario.
POST	<code>/api/auth/login</code>	Público	Autentica usuario. Devuelve token JWT + mecanicold/clienteld según rol.
PATCH	<code>/api/auth/foto-perfil</code>	Autenticado	Actualiza la foto de perfil del usuario en formato Base64 (LONGTEXT en BD).
POST	<code>/api/admin/setup</code>	Público	Crea el primer administrador. Devuelve 409 si ya existe alguno.

Módulo Administrador (`/api/admin`) — Requiere rol ADMIN

Método	Ruta	Descripción
GET / POST / PUT / DELETE	<code>/api/admin/mecanicos</code> , <code>/api/admin/mecanicos/{id}</code>	CRUD completo. El POST crea también el usuario con rol MECANICO y la contraseña hasheada.

Método	Ruta	Descripción
GET / POST / PUT / DELETE	/api/admin/clientes, /api/admin/clientes/{id}	CRUD completo. El POST crea el usuario con rol CLIENTE.
GET / POST / PUT / DELETE	/api/admin/vehiculos, /api/admin/vehiculos/{id}	CRUD completo de vehículos con asignación a cliente.
GET / POST / DELETE	/api/admin/ordenes, /api/admin/ordenes/{id}	Gestión de órdenes con notificación automática al crear.
PATCH	/api/admin/ordenes/{id}/estado	Cambia el estado con observación. Genera notificaciones a mecánico y cliente.
PUT	/api/admin/ordenes/{id}	Actualiza diagnóstico, precios y fechas. Genera notificación al cliente si cambia el diagnóstico.
GET	/api/admin/ordenes/{id}/historial	Historial completo de cambios de estado y diagnóstico de una orden.
GET / POST / DELETE	/api/admin/horarios, /api/admin/horarios/{id}	Gestión de horarios con notificación automática al mecánico al crear o eliminar.
GET / PATCH	/api/admin/ausencias, /api/admin/ausencias/{id}	Listado y aprobación/rechazo con notificación automática al mecánico.
GET / POST / PUT / DELETE	/api/admin/plazas, /api/admin/plazas/{id}	CRUD completo de plazas de estacionamiento.

Módulo Mecánico (/api/mecanico) — Requiere rol MECANICO o ADMIN

Método	Ruta	Descripción
GET	/api/mecanico/ordenes/{mecanicoId}	Órdenes asignadas al mecánico.
PATCH	/api/mecanico/ordenes/{id}/estado	Cambia el estado de una orden. Genera notificaciones.
PATCH	/api/mecanico/ordenes/{id}/diagnostico	Actualiza el diagnóstico y guarda en historial. Notifica al cliente.
GET	/api/mecanico/ordenes/{id}/historial	Historial de cambios de una orden.
GET	/api/mecanico/horarios/{mecanicoId}	Horarios semanales del mecánico.
GET	/api/mecanico/ausencias/{mecanicoId}	Solicitudes de ausencia del mecánico.
POST	/api/mecanico/ausencias	Solicita una nueva ausencia. Notifica a todos los administradores.

Módulo Cliente (/api/cliente) — Requiere rol CLIENTE o ADMIN

Método	Ruta	Descripción
GET	/api/cliente/vehiculos/{clienteId}	Vehículos registrados a nombre del cliente.
GET	/api/cliente/ordenes/{clienteId}	Órdenes de trabajo del cliente.
GET	/api/cliente/ordenes/{id}/historial	Historial de cambios de una orden.

Módulo Notificaciones (/api/notificaciones) — Cualquier usuario autenticado

Método	Ruta	Descripción
GET	/api/notificaciones	Todas las notificaciones del usuario autenticado, ordenadas por fecha descendente.
GET	/api/notificaciones/no-leidas	Número de notificaciones no leídas del usuario autenticado.
PATCH	/api/notificaciones/{id}/leer	Marca una notificación específica como leída.
PATCH	/api/notificaciones/leer-todas	Marca todas las notificaciones del usuario como leídas.

2.3.5. Sistema de Notificaciones Automáticas

El sistema genera notificaciones automáticamente ante los siguientes eventos, sin necesidad de intervención manual:

Evento	Destinatario(s)	Título	Tipo
Creación de orden de trabajo	Mecánico asignado	Nueva orden asignada	GENERAL
Creación de orden de trabajo	Cliente propietario del vehículo	Orden de trabajo creada	GENERAL
Cambio de estado de orden	Cliente propietario	Actualización de tu vehículo	ESTADO
Cambio de estado de orden	Mecánico asignado	Estado actualizado — Orden #X	ESTADO
Actualización de diagnóstico	Cliente propietario	Diagnóstico actualizado	ESTADO
Creación de horario	Mecánico afectado	Nuevo horario asignado	GENERAL
Eliminación de horario	Mecánico afectado	Horario eliminado	GENERAL
Solicitud de ausencia	Todos los administradores	Nueva solicitud de ausencia	AUSENCIA
Aprobación de ausencia	Mecánico solicitante	Ausencia aprobada	AUSENCIA
Rechazo de ausencia	Mecánico solicitante	Ausencia rechazada	AUSENCIA
Scheduler diario 8:00h	Todos los administradores	Ausencia próxima — Mecánico X	AUSENCIA

El scheduler de ausencias próximas se implementa mediante la anotación `@Scheduled` con expresión cron `'0 0 8 * * *'`. Ejecuta diariamente a las 8:00h y avisa al administrador de todas las ausencias aprobadas que comiencen en los próximos 1 o 2 días, indicando el nombre del mecánico, las fechas de inicio y fin y el tipo de ausencia.

2.3.6. Gestión Centralizada de Errores

La clase `GlobalExceptionHandler` (anotada con `@RestControllerAdvice`) intercepta todas las excepciones no controladas y las convierte en respuestas HTTP con mensajes descriptivos y comprensibles para el usuario. Los errores gestionados son:

Excepción	Código HTTP	Mensaje devuelto
ResourceNotFoundException	404 Not Found	Mensaje descriptivo del recurso no encontrado
BadCredentialsException	401 Unauthorized	Credenciales incorrectas. Comprueba tu email y contraseña.
AccessDeniedException	403 Forbidden	No tienes permiso para realizar esta acción.
DataIntegrityViolationException (email)	409 Conflict	Ya existe un usuario registrado con ese correo electrónico.
DataIntegrityViolationException (matrícula)	409 Conflict	Ya existe un vehículo registrado con esa matrícula.
DataIntegrityViolationException (num_empleado)	409 Conflict	Ya existe un mecánico con ese número de empleado.
IllegalArgumentException (enum inválido)	400 Bad Request	Valor no válido para el campo seleccionado.
Exception genérica	500 Internal Server Error	Ha ocurrido un error inesperado. Por favor, inténtalo de nuevo.

2.4. Aplicación de Escritorio — JavaFX

2.4.1. Stack Tecnológico

Componente	Versión	Uso
Java	21 (LTS)	Lenguaje de programación
JavaFX	21.0.6	Framework de interfaz gráfica
Maven	3.x	Gestión de dependencias y build
OkHttp3	4.12.0	Cliente HTTP para llamadas a la API REST
Jackson	2.17.0	Serialización/deserialización JSON
ControlsFX	11.2.1	Componentes adicionales de JavaFX

El fichero module-info.java declara los módulos necesarios: javafx.controls, javafx.fxml, org.controlsfx.controls, com.fasterxml.jackson.databind y okhttp3.

2.4.2. Estructura del Proyecto

Paquete/Archivo	Descripción
controller/LoginController	Gestión del login con toggle de contraseña dinámico (dos campos intercambiables sin rectángulos visibles).
controller/DashboardController	Controlador principal que gestiona la navegación entre vistas mediante el sidebar.
controller/HomeController	Dashboard: carga en hilos paralelos las tarjetas de resumen, tabla de órdenes, mapa de plazas y notificaciones recientes.
controller/MecanicosController	CRUD de mecánicos con tabla, buscador, formulario con validaciones y llamadas a la API.

Paquete/Archivo	Descripción
controller/CientesController	CRUD de clientes con validación de NIF español (algoritmo de la letra).
controller/VehiculosController	CRUD de vehículos con validación de matrícula española y ComboBox de clientes.
controller/OrdenesController	CRUD de órdenes: crear, ver detalle, cambiar estado (con validación de estado diferente) y eliminar.
controller/HorariosController	Gestión de horarios con filtro por mecánico y validación de horas.
controller/AusenciasController	Aprobación y rechazo de solicitudes de ausencia.
controller/PlazasController	Gestión de plazas con mapa visual en tiempo real.
service/AuthService	Gestiona el login contra la API, almacena el token JWT y los datos del usuario. Método login devuelve null en éxito o mensaje de error como String.
view/*.fxml	Ficheros FXML para cada vista: login, dashboard, home, mecánicos, clientes, vehículos, órdenes, horarios, ausencias, plazas.
resources/styles.css	Hoja de estilos CSS global con el tema oscuro personalizado.

2.4.3. Patrones de Implementación

La aplicación sigue estos patrones consistentes en todos los controladores:

- Todas las llamadas a la API REST se realizan en hilos separados (`new Thread(() -> {...}).start()`) para no bloquear el hilo de la interfaz gráfica de JavaFX.
- Las actualizaciones de la interfaz gráfica desde hilos secundarios se realizan siempre mediante `Platform.runLater(() -> {...})`.
- Las tablas usan `Map<String, String>` como modelo de datos con lambdas `SimpleStringProperty` en lugar de `PropertyValueFactory`, por compatibilidad con el tipo `Map`.
- Los formularios usan `Dialog<Map<String, String>>` con `GridPane`. Las validaciones se implementan mediante `addEventFilter` en el botón OK con `event.consume()` para evitar que el diálogo se cierre al detectar errores.
- Los `ComboBox` usan `setButtonCell` con `ListCell` personalizado para mostrar el texto en blanco sobre fondo oscuro.
- Las `TextArea` usan `-fx-control-inner-background: #252545` para el fondo oscuro y `-fx-prompt-text-fill: #555575` para el texto de marcador.

2.4.4. Validaciones en Formularios

Campo	Validación	Mensaje de error
Email	Expresión regular: <code>^[^s@]+@[^s@]+\.[^s@]+\$</code>	El correo electrónico no es válido.
Contraseña (nueva)	Mínimo 6 caracteres	La contraseña debe tener al menos 6 caracteres.
Teléfono	Expresión regular: <code>^[6-9][0-9]{8}\$</code>	El teléfono no es válido. Debe tener 9 dígitos y empezar por 6, 7, 8 o 9.
NIF	Formato 8 dígitos + letra. Algoritmo de la letra (módulo	El NIF no es válido. La letra no corresponde al número.

Campo	Validación	Mensaje de error
	23 sobre la cadena TRWAGMYFPDXBNJZSQVHLCKE)	
Matrícula	Expresión regular: $^{[0-9]{4}}[A-Za-z]{3}\$$	La matrícula no es válida. Formato esperado: 1234ABC.
Año vehículo	Número entre 1900 y 2026	El año debe estar entre 1900 y 2026.
Hora horario	Expresión regular: $^{([01]?[0-9] 2[0-3]):[0-5][0-9]\$$	La hora no es válida. Formato esperado: HH:mm.
Lógica de horas	hora_entrada debe ser anterior a hora_salida	La hora de entrada debe ser anterior a la hora de salida.
Descripción de orden	Mínimo 10 caracteres	La descripción debe tener al menos 10 caracteres.
Años de experiencia	Número entre 0 y 50	Los años de experiencia deben estar entre 0 y 50.

2.4.5. Capturas de Pantalla

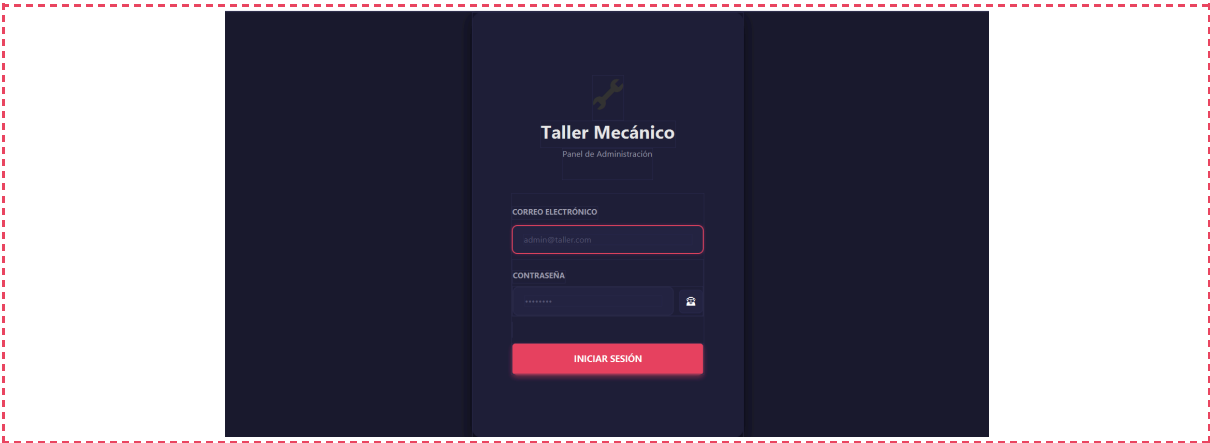


Figura 3: Pantalla de login de la aplicación de escritorio

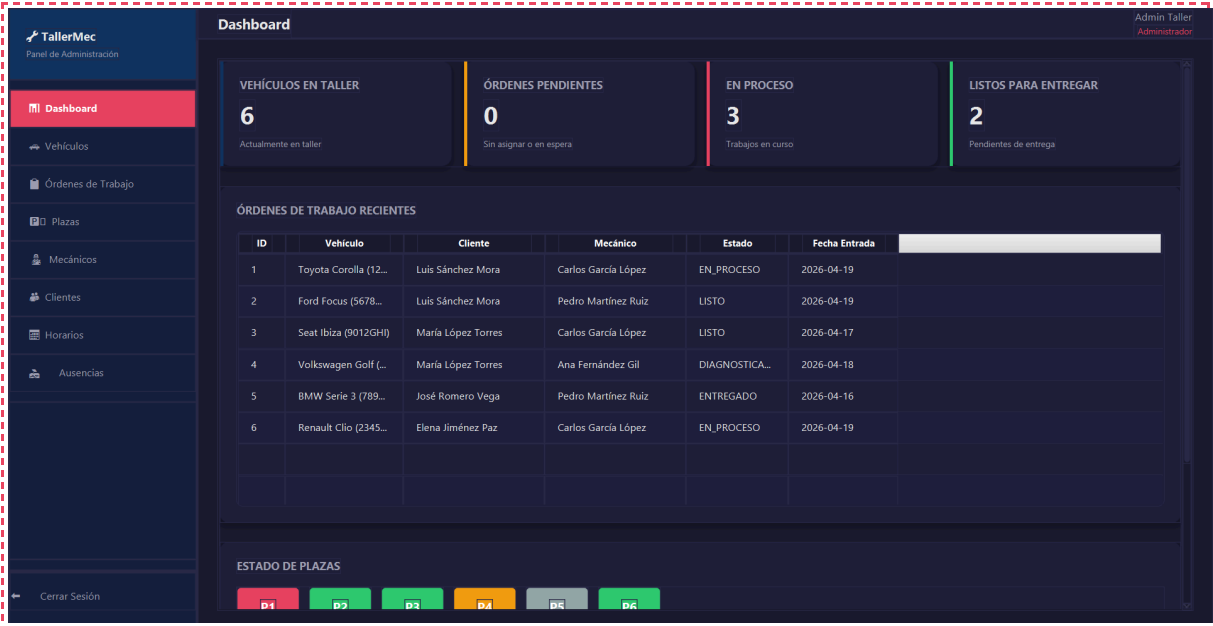


Figura 4: Dashboard principal con resumen en tiempo real y notificaciones

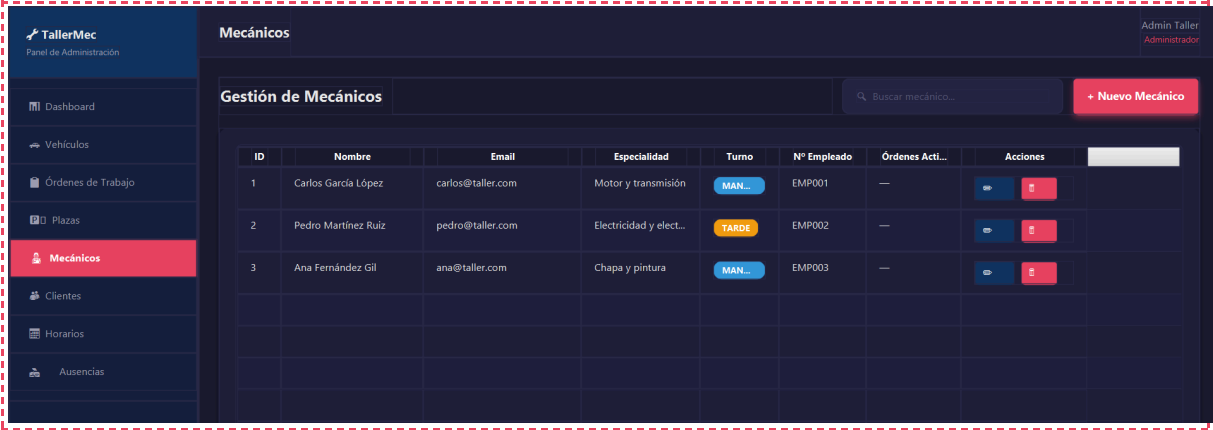


Figura 5: Gestión de mecánicos

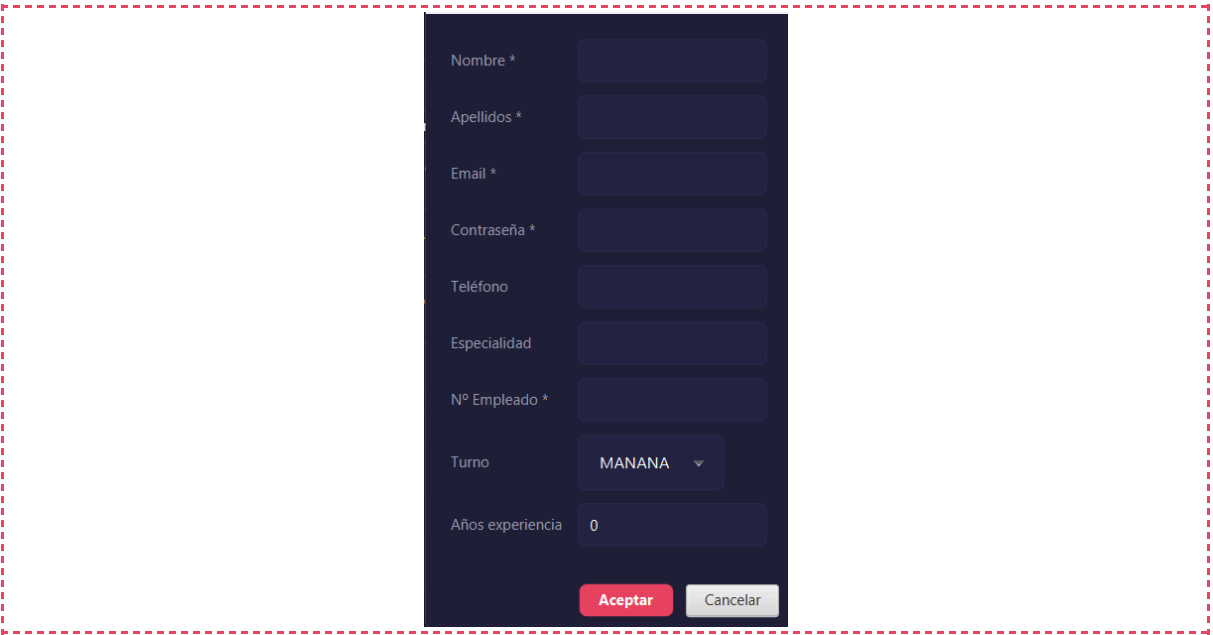


Figura 6: Formulario de creación de mecánico

TallerMec

Panel de Administración

Dashboard

Vehículos

Órdenes de Trabajo

Plazas

Mecánicos

Clientes

Horarios

Ausencias

Órdenes de Trabajo

TODOS

Buscar...

Nueva Orden

ID	Vehículo	Cliente	Mecánico	Estado	Precio Est.	Fecha Entrada	Acciones
1	Toyota Corolla (123...	Luis Sánchez Mora	Carlos García López	EN_PROCESO	450.0 €	2026-04-19	
2	Ford Focus (5678DEF)	Luis Sánchez Mora	Pedro Martínez Ruiz	LISTO	80.0 €	2026-04-19	
3	Seat Ibiza (9012GHI)	María López Torres	Carlos García López	LISTO	220.0 €	2026-04-17	
4	Volkswagen Golf (3...	María López Torres	Ana Fernández Gil	DIAGNOSTI...	800.0 €	2026-04-18	
5	BMW Serie 3 (7890...	José Romero Vega	Pedro Martínez Ruiz	ENTREGADO	350.0 €	2026-04-16	
6	Renault Clio (2345P...	Elena Jiménez Paz	Carlos García López	EN_PROCESO	120.0 €	2026-04-19	

Figura 7: Gestión de órdenes de trabajo

Vehículo

Cliente

Mecánico

Estado

Descripción

Diagnóstico

Precio estimado

Precio final

Fecha entrada

Notas internas

Toyota Corolla (1234ABC)

Luis Sánchez Mora

Carlos García López

EN_PROCESO

El motor hace ruido extraño al arrancar

Fallo en la correa de distribución

450.0 €

—

2026-04-19

—

Cerrar

Figura 8: Detalle de orden de trabajo

Nuevo estado *

Observación

EN_PROCESO

Observación (opcional)

Aceptar

Cancelar

Figura 9: Diálogo de cambio de estado

TallerMec

Panel de Administración

Dashboard

Vehículos

Órdenes de Trabajo

Plazas

Mecánicos

Clientes

Horarios

Ausencias

Horarios

Admin Taller

Administrador

Gestión de Horarios

TODOS

+ Añadir Horario

ID	Mecánico	Día	Hora Entrada	Hora Salida	Acciones
1	Carlos García López	LUNES	08:00:00	16:00:00	
2	Carlos García López	MARTES	08:00:00	16:00:00	
3	Carlos García López	MIÉRCOLES	08:00:00	16:00:00	
4	Carlos García López	JUEVES	08:00:00	16:00:00	
5	Carlos García López	VIERNES	08:00:00	16:00:00	
6	Pedro Martínez Ruiz	LUNES	14:00:00	22:00:00	
7	Pedro Martínez Ruiz	MARTES	14:00:00	22:00:00	
8	Pedro Martínez Ruiz	MIÉRCOLES	14:00:00	22:00:00	

Figura 10: Gestión de horarios

TallerMec

Panel de Administración

Dashboard

Vehículos

Órdenes de Trabajo

Plazas

Mecánicos

Clientes

Horarios

Ausencias

Ausencias

Admin Taller

Administrador

Gestión de Ausencias

TODAS

ID	Mecánico	Tipo	Fecha Inicio	Fecha Fin	Motivo	Estado	Acciones
1	Carlos García López	VACACIONES	2026-05-01	2026-05-07	Vacaciones de primavera	APROBADA	
2	Pedro Martínez Ruiz	BAIA	2026-04-20	2026-04-22	Baja médica	PENDIENTE	
3	Ana Fernández Gil	OTRO	2026-04-25	2026-04-25	Asunto personal	PENDIENTE	
4	Carlos García López	VACACIONES	2026-05-02	2026-05-05	Lo que sea	PENDIENTE	

Figura 11: Gestión de ausencias

2.5. Aplicación Móvil — React Native

2.5.1. Stack Tecnológico

Componente	Versión	Uso
React Native	0.76	Framework de desarrollo móvil multiplataforma
Expo	SDK 54	Herramientas de desarrollo y build
TypeScript	5.x	Tipado estático del código JavaScript
React Navigation	6.x	Navegación entre pantallas y pestañas
Axios	1.x	Cliente HTTP con interceptores JWT
AsyncStorage	1.x	Persistencia local del token y datos de usuario
expo-image-picker	16.x	Selección de foto de perfil desde la galería

2.5.2. Estructura del Proyecto

Archivo/Carpeta	Descripción
src/api/api.ts	Instancia Axios con URL base (http://10.0.2.2:8080/api para emulador). Interceptor de petición: añade el token JWT. Interceptor de respuesta: extrae el mensaje del backend o asigna mensajes genéricos según el código HTTP. Limpia el token al recibir 401.
src/store/authStore.ts	Funciones saveAuth (guarda token + user en AsyncStorage), getToken, getUser y clearAuth.
src/utils/fecha.ts	formatFecha: convierte YYYY-MM-DD a DD/MM/YYYY. formatFechaHora: convierte YYYY-MM-DDTHH:mm a DD/MM/YYYY HH:mm.
src/utils/validaciones.ts	Funciones: validarEmail (regex), validarNIF (formato + algoritmo letra), validarMatricula (regex), validarTelefono (regex).
src/navigation/AppNavigator.tsx	Detecta el rol del usuario tras el login y redirige a MecanicoNavigator o ClienteNavigator.
src/navigation/MecanicoNavigator.tsx	Cinco pestañas: Órdenes, Horarios, Ausencias, Notificaciones (con badge), Perfil. Polling de notificaciones cada 30 segundos.
src/navigation/ClienteNavigator.tsx	Cuatro pestañas: Mis Vehículos, Mis Órdenes, Notificaciones (con badge), Perfil. Polling cada 30 segundos.
src/screens/auth/LoginScreen.tsx	Login con toggle de contraseña y enlace al registro.
src/screens/auth/RegisterScreen.tsx	Registro de clientes con validación de email, contraseña mínima 6 caracteres y teléfono.
src/screens/mecanico/MecanicoOrdenesScreen.tsx	Lista de órdenes, detalle con historial y modal de actualización de estado/diagnóstico.
src/screens/mecanico/MecanicoHorariosScreen.tsx	Horarios semanales con badges de día.
src/screens/mecanico/MecanicoAusenciasScreen.tsx	Lista de ausencias y formulario de solicitud con validación de fechas DD/MM/AAAA.
src/screens/mecanico/MecanicoPerfilScreen.tsx	Perfil con foto de galería, opción de eliminar foto y cierre de sesión.
src/screens/cliente/ClienteVehiculosScreen.tsx	Vehículos del cliente.
src/screens/cliente/ClienteOrdenesScreen.tsx	Lista de órdenes y detalle con estado descriptivo, historial e información del mecánico.
src/screens/cliente/ClientePerfilScreen.tsx	Perfil con foto de galería y cierre de sesión.
src/screens/shared/NotificacionesScreen.tsx	Notificaciones con badge de no leídas, marcar individual/todas como leídas.

2.5.3. Funcionalidades Detalladas

Sistema de notificaciones en la app móvil

El badge de notificaciones no leídas se implementa en MecanicoNavigator y ClienteNavigator mediante un estado noLeidas que se actualiza llamando a GET /api/notificaciones/no-leidas. Esta llamada se realiza al montar el componente y cada 30 segundos mediante setInterval en un useEffect. El badge se muestra como un círculo rojo sobre el icono de la campana con el número de mensajes pendientes (o '9+' si son más de 9).

Historial de órdenes

Al pulsar sobre una orden, se carga el historial mediante GET /api/mecanico/ordenes/{id}/historial (para mecánicos) o GET /api/cliente/ordenes/{id}/historial (para clientes). El historial incluye cambios de estado (con estado anterior y nuevo) y cambios de diagnóstico (identificados por estadoNuevo === 'DIAGNOSTICO'), cada uno con fecha formateada en DD/MM/YYYY HH:mm.

Foto de perfil

Se usa expo-image-picker para solicitar permiso de acceso a la galería y seleccionar una imagen cuadrada (aspect [1,1]) con calidad 0.5 y opción base64 activada. La imagen se convierte a cadena data:image/jpeg;base64,{base64} y se envía al endpoint PATCH /api/auth/foto-perfil. Se guarda en la columna foto_perfil de la tabla USUARIO como LONGTEXT en la base de datos. También se actualiza en AsyncStorage para que persista localmente.

Solicitud de ausencias

El formulario acepta fechas en formato DD/MM/AAAA. Antes de enviar al backend, la función convertirFecha transforma el formato a YYYY-MM-DD (requerido por la API). La validación comprueba que ambas fechas tengan el formato correcto y que la fecha de inicio no sea posterior a la fecha de fin.

2.5.4. Capturas de Pantalla

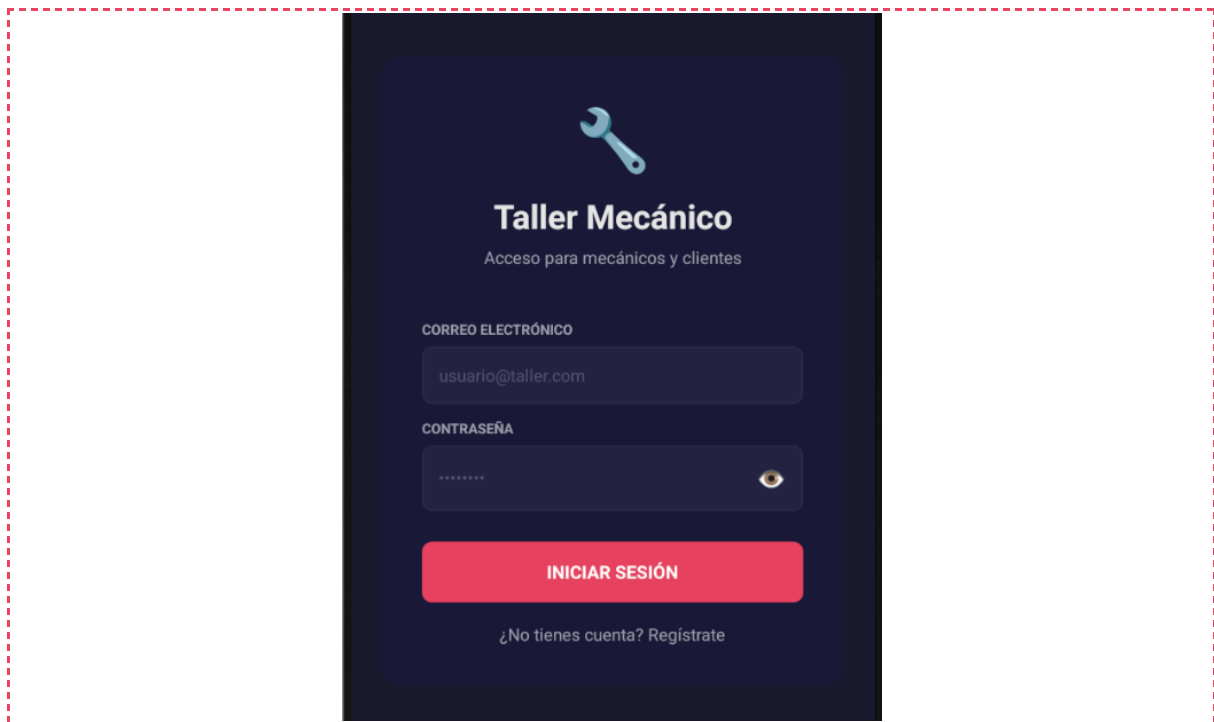


Figura 12: Pantalla de login de la aplicación móvil

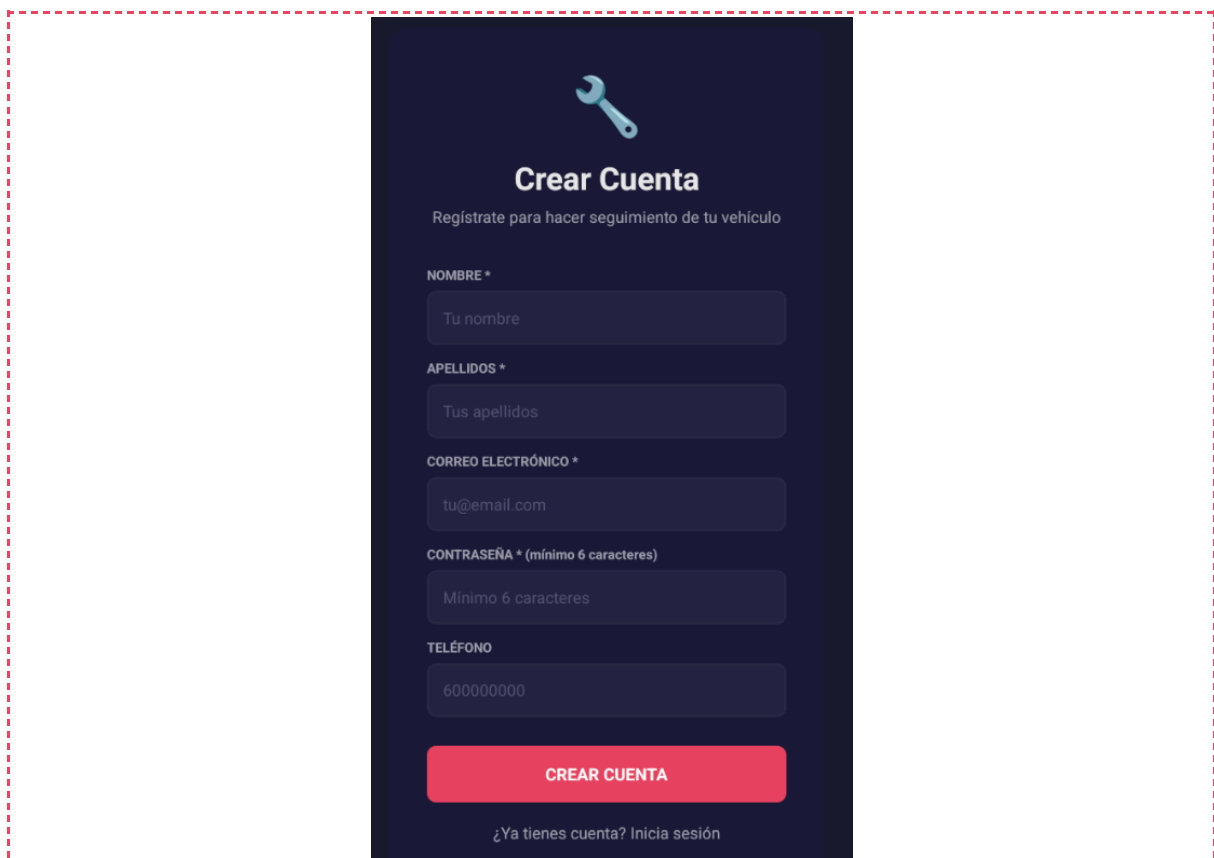


Figura 13: Pantalla de registro de clientes

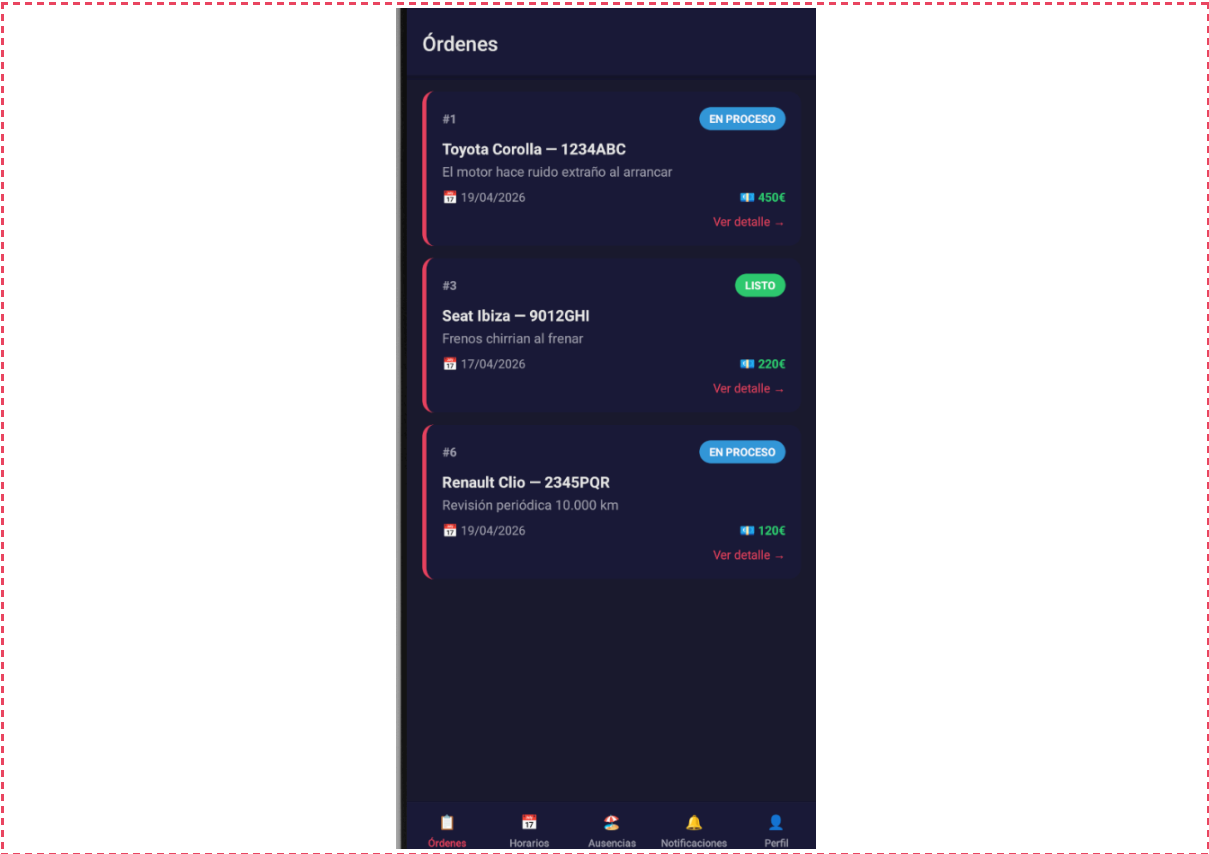


Figura 14: Órdenes del mecánico

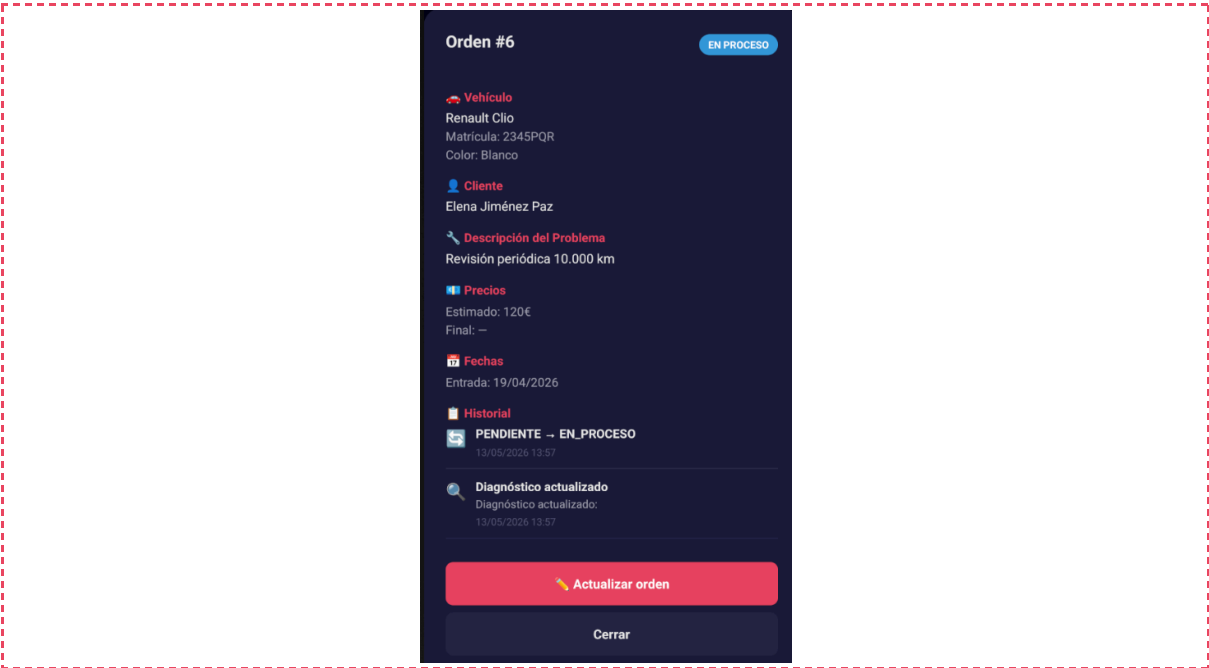


Figura 15: Detalle de orden con historial



Actualizar Orden #6

Estado

PENDIENTE **EN PROCESO** DIAGNOSTICADO

REPARADO LISTO ENTREGADO

Diagnóstico

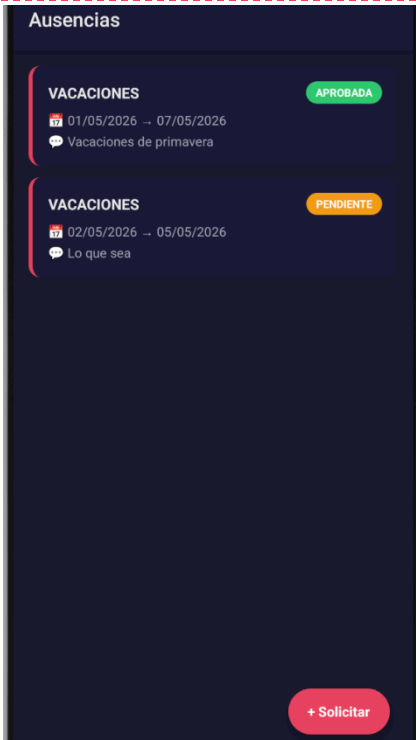
Escribe el diagnóstico...

Observación (opcional)

Observación sobre el cambio...

Cancelar Guardar

Figura 16: Modal de actualización de estado y diagnóstico



Ausencias

VACACIONES APROBADA

01/05/2026 - 07/05/2026

Vacaciones de primavera

VACACIONES PENDIENTE

02/05/2026 - 05/05/2026

Lo que sea

+ Solicitar

Figura 17: Ausencias del mecánico

Solicitar Ausencia

TIPO

VACACIONES

DIA_LIBRE

BAJA

OTRO

FECHA INICIO *

DD/MM/AAAA

FECHA FIN *

DD/MM/AAAA

MOTIVO

Describe el motivo (opcional)

Cancelar

Enviar

Figura 18: Formulario de solicitud de ausencia

Horarios

LUNES

Entrada: 08:00:00

Salida: 16:00:00

MARTES

Entrada: 08:00:00

Salida: 16:00:00

MIERCOLES

Entrada: 08:00:00

Salida: 16:00:00

JUEVES

Entrada: 08:00:00

Salida: 16:00:00

VIERNES

Entrada: 08:00:00

Salida: 16:00:00

Figura 19: Horarios del mecánico

Notificaciones

Ausencia rechazada

AUSENCIA

Tu solicitud de ausencia del 2026-05-05 al 2026-05-07 ha sido rechazada.

16/05/2026 01:46

Estado actualizado — Orden #6

ESTADO

La orden #6 ha cambiado a: En proceso de reparación

13/05/2026 13:57

Figura 20: Centro de notificaciones

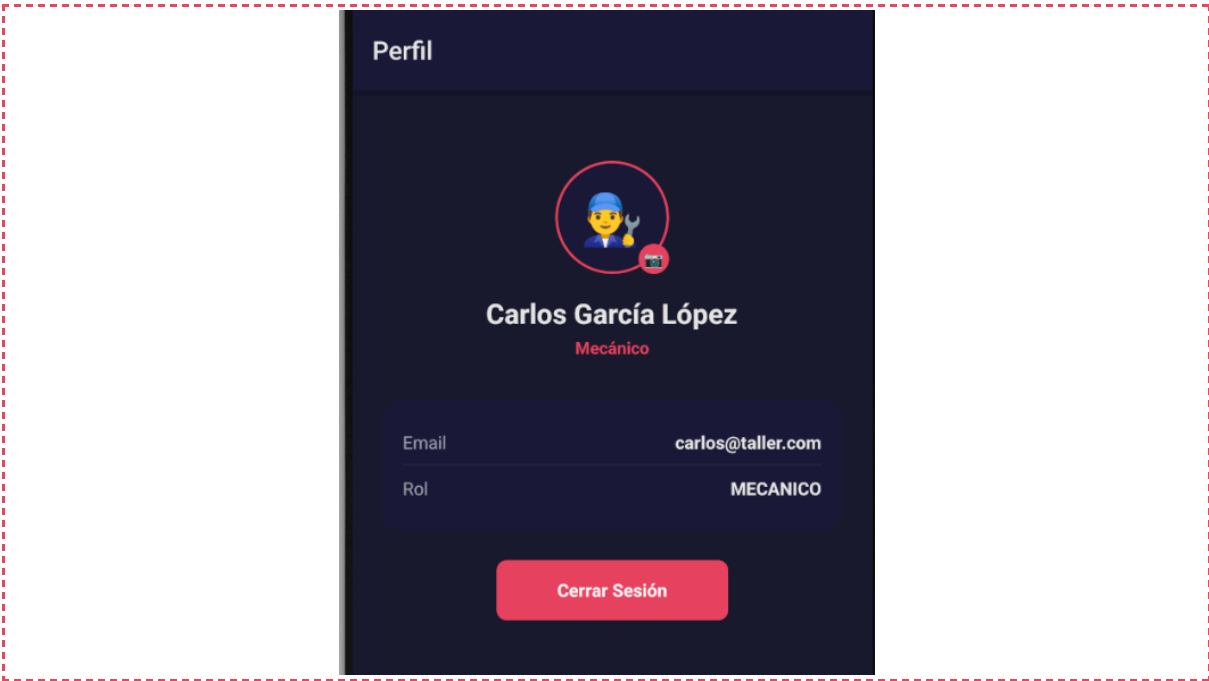


Figura 21: Perfil del mecánico

2.6. Diseño Estético

Ambas aplicaciones comparten una identidad visual coherente basada en un tema oscuro profesional:

Color	Código HEX	Uso
Fondo principal	#1A1A2E	Fondo general de la aplicación
Fondo de tarjetas	#1E1E3A	Cards, modales y paneles
Fondo de campos	#252545	Inputs, TextAreas, ComboBoxes
Acento rojo	#E94560	Botón primario, bordes activos, badges PENDIENTE, badges no leídas
Acento azul oscuro	#0F3460	Sidebar activo, botones secundarios, badge MAÑANA
Acento azul medio	#16213E	Sidebar inactivo, cabeceras de tabla
Texto principal	#EAEAEA	Texto principal sobre fondos oscuros
Texto secundario	#A0A0B0	Etiquetas, subtítulos, texto atenuado
Texto placeholder	#555575	Texto de marcador en campos de entrada
Éxito (verde)	#2ECC71	Estado LISTO, precios finales, aprobado
Advertencia (naranja)	#F39C12	Estado PENDIENTE, precio estimado, badge TARDE
Púrpura	#9B59B6	Estado DIAGNOSTICADO, badge NOCHE
Turquesa	#1ABC9C	Estado REPARADO
Gris	#555575	Estado ENTREGADO, notificaciones leídas

Los estados de las órdenes de trabajo tienen un color específico asignado tanto en la app de escritorio (badges en la tabla) como en la app móvil (badges en tarjetas), garantizando consistencia visual entre plataformas.

La tipografía utilizada es Segoe UI en la app de escritorio y el sistema sans-serif nativo en la app móvil. Los formularios siguen un patrón uniforme: fondo #1E1E3A, campos #252545 con borde #2A2A4A, texto blanco y etiquetas en #A0A0B0.

3. FASE DE PRUEBAS

3.1. Pruebas del Backend (API REST)

Las pruebas del backend se han realizado mediante Postman, verificando el correcto funcionamiento de todos los endpoints. Se han ejecutado los siguientes casos de prueba:

Caso de prueba	Acción	Resultado esperado	Resultado obtenido
Login con credenciales correctas	POST /api/auth/login con email y password válidos	200 OK con token JWT y datos del usuario	CORRECTO
Login con credenciales incorrectas	POST /api/auth/login con password incorrecta	401 con mensaje 'Credenciales incorrectas'	CORRECTO
Acceso a ruta de admin sin token	GET /api/admin/mecanicos sin cabecera Authorization	401 Unauthorized	CORRECTO
Acceso a ruta de admin con token de mecánico	GET /api/admin/mecanicos con token de MECANICO	403 Forbidden	CORRECTO
Crear mecánico con email duplicado	POST /api/admin/mecanicos con email existente	409 con mensaje descriptivo de email duplicado	CORRECTO
Crear vehículo con matrícula duplicada	POST /api/admin/vehiculos con matrícula existente	409 con mensaje descriptivo de matrícula duplicada	CORRECTO
Cambiar estado de orden	PATCH /api/admin/ordenes/{id}/estado	200 OK con la orden actualizada y notificaciones generadas	CORRECTO
Actualizar diagnóstico	PATCH /api/mecanico/ordenes/{id}/diagnostico	200 OK, historial actualizado y notificación al cliente	CORRECTO
Solicitar ausencia	POST /api/mecanico/ausencias	200 OK y notificación al administrador	CORRECTO
Aprobar ausencia	PATCH /api/admin/ausencias/{id} con aprobada=true	200 OK y notificación al mecánico	CORRECTO
Setup con admin existente	POST /api/admin/setup cuando ya existe un ADMIN	409 con mensaje 'Ya existe un administrador'	CORRECTO
Subir foto de perfil	PATCH /api/auth/foto-perfil con Base64 de imagen	204 No Content, foto guardada en BD como LONGTEXT	CORRECTO

3.2. Pruebas de la Aplicación de Escritorio

Caso de prueba	Acción	Resultado esperado	Resultado obtenido
Login con credenciales correctas	Introducir email y password válidos y pulsar INICIAR SESIÓN	Redirige al dashboard	CORRECTO
Login con credenciales incorrectas	Introducir password incorrecta	Muestra mensaje de error sin cerrar el formulario	CORRECTO
Toggle de contraseña en login	Pulsar botón Ver/Ocultar en el campo de contraseña	Muestra/oculta la contraseña sin rectángulos extra	CORRECTO
Crear mecánico con datos válidos	Rellenar formulario y pulsar Aceptar	Mecánico creado y tabla actualizada	CORRECTO
Crear mecánico con email inválido	Introducir email sin '@' y pulsar Aceptar	Formulario permanece abierto con mensaje de error	CORRECTO
Crear cliente con NIF incorrecto	Introducir NIF con letra errónea y pulsar Aceptar	Formulario permanece abierto con mensaje de error	CORRECTO
Crear vehículo con matrícula inválida	Introducir matrícula sin formato correcto y pulsar Aceptar	Formulario permanece abierto con mensaje de error	CORRECTO
Crear horario con horas inválidas	Introducir hora salida anterior a hora entrada	Formulario permanece abierto con mensaje de error	CORRECTO
Cambiar estado a mismo estado	Seleccionar el mismo estado actual y pulsar Aceptar	Formulario permanece abierto con aviso	CORRECTO
Ver notificaciones en dashboard	Generar un cambio de estado desde otra plataforma y recargar el dashboard	Aparece la notificación en el panel	CORRECTO
Aprobar ausencia	Pulsar Aprobar en una solicitud pendiente	Estado cambia a aprobada y mecánico recibe notificación	CORRECTO
Eliminar mecánico	Pulsar el icono de eliminar y confirmar	Mecánico eliminado y tabla actualizada	CORRECTO

3.3. Pruebas de la Aplicación Móvil

Caso de prueba	Acción	Resultado esperado	Resultado obtenido
Registro de nuevo cliente	Rellenar formulario y pulsar CREAR CUENTA	Cuenta creada y redirección al login	CORRECTO

Caso de prueba	Acción	Resultado esperado	Resultado obtenido
Registro con email inválido	Introducir email sin '@'	Alert con mensaje de error específico	CORRECTO
Login con rol de mecánico	Login con carlos@taller.com	Redirige al panel del mecánico con 5 pestañas	CORRECTO
Login con rol de cliente	Login con luis@cliente.com	Redirige al panel del cliente con 4 pestañas	CORRECTO
Ver detalle de orden (mecánico)	Pulsar sobre una orden en la lista	Muestra detalle completo con historial de cambios	CORRECTO
Actualizar estado de orden	Pulsar 'Actualizar orden', cambiar estado y guardar	Estado actualizado y notificaciones enviadas	CORRECTO
Actualizar diagnóstico	Modificar el diagnóstico en el modal y guardar	Diagnóstico actualizado y historial registrado	CORRECTO
Solicitar ausencia con fecha válida	Introducir fecha en formato DD/MM/AAAA y enviar	Ausencia creada con estado PENDIENTE	CORRECTO
Solicitar ausencia con fecha inválida	Introducir fecha con formato incorrecto	Alert con mensaje de error de formato	CORRECTO
Recibir notificación	Generar un cambio de estado desde el escritorio	Badge aparece en la pestaña de notificaciones	CORRECTO
Marcar notificación como leída	Pulsar sobre una notificación no leída	Notificación pasa a estado leído (color atenuado)	CORRECTO
Seleccionar foto de perfil	Pulsar el avatar y seleccionar imagen de la galería	Foto actualizada y guardada en servidor	CORRECTO
Eliminar foto de perfil	Pulsar 'Eliminar foto' y confirmar	Avatar vuelve al icono por defecto	CORRECTO
Ver detalle de orden (cliente)	Pulsar sobre una orden en la lista del cliente	Muestra estado descriptivo, mecánico e historial	CORRECTO

3.4. Pruebas de Integración

Las pruebas de integración verifican que las tres aplicaciones trabajan correctamente con los mismos datos:

Flujo de integración	Pasos	Resultado
Creación de orden y notificación	1. Admin crea una orden en escritorio. 2. Mecánico abre la app móvil. 3. Badge de notificaciones muestra 1 no leída.	CORRECTO

Flujo de integración	Pasos	Resultado
Actualización de estado multi-plataforma	1. Mecánico cambia estado en móvil a 'LISTO'. 2. Cliente recibe notificación en móvil. 3. Admin ve el cambio en escritorio.	CORRECTO
Solicitud y aprobación de ausencia	1. Mecánico solicita ausencia en móvil. 2. Admin recibe notificación en dashboard. 3. Admin aprueba en escritorio. 4. Mecánico recibe notificación de aprobación.	CORRECTO
Registro de cliente y acceso	1. Nuevo cliente se registra en app móvil. 2. Admin puede ver y editar el cliente en escritorio. 3. Cliente accede a su panel.	CORRECTO

4. DOCUMENTACIÓN DE LA APLICACIÓN

4.1. Manual de Instalación

4.1.1. Requisitos del Sistema

Componente	Requisito mínimo	Notas
Java (backend)	JDK 24 o superior	Requerido para ejecutar Spring Boot 3.5
Java (escritorio)	JDK 21 o superior	JDK 21 LTS recomendado para JavaFX
Node.js	18.x o superior	Para la app móvil React Native + Expo
MySQL	8.x	Gestionado mediante XAMPP + phpMyAdmin
Android	API 26 (Android 8.0) o superior	Para la app móvil en dispositivo o emulador
RAM	8 GB mínimo	Para ejecutar IntelliJ, Android Studio y el emulador simultáneamente
Espacio en disco	10 GB libres	Para las herramientas de desarrollo y los emuladores

4.1.2. Instalación de la Base de Datos

1. Instalar XAMPP e iniciar los servicios Apache y MySQL.
2. Acceder a phpMyAdmin en <http://localhost/phpmyadmin>.
3. Crear una nueva base de datos con el nombre: taller_mecanico.
4. Importar el SQL incluido en el repositorio.

4.1.3. Instalación y arranque del Backend

1. Clonar el repositorio del proyecto.
2. Abrir la carpeta taller-backend en IntelliJ IDEA.
3. Verificar que application.yml contiene la configuración correcta de la base de datos:
datasource.url: jdbc:mysql://localhost:3306/taller_mecanico
datasource.username: root | datasource.password: (vacío en XAMPP por defecto)
4. Ejecutar la clase TallerBackendApplication.java desde IntelliJ (NO mediante Maven).
5. El servidor arranca en <http://localhost:8080>.
6. Crear el primer administrador llamando al endpoint de setup:
POST <http://localhost:8080/api/admin/setup>
Content-Type: application/json
Body: { "nombre": "Admin", "apellidos": "Taller", "email": "admin@taller.com", "password": "admin123" }
Una vez creado el primer administrador, este endpoint queda bloqueado automáticamente.

4.1.4. Instalación y arranque de la App de Escritorio

1. Abrir la carpeta taller-desktop en IntelliJ IDEA.
2. Asegurarse de que el backend está en ejecución.
3. Ejecutar la clase MainApp.java desde IntelliJ IDEA.
4. La aplicación se abre en la pantalla de login.

4.1.5. Instalación y arranque de la App Móvil

1. Abrir la carpeta taller-mobile en Visual Studio Code.
2. Instalar las dependencias: npm install
3. Iniciar el emulador Android desde Android Studio (Pixel 6, API 34 recomendado).
4. Ejecutar: npx expo start
5. Pulsar 'a' para abrir en el emulador Android.

Nota: la URL base de la API está configurada como `http://10.0.2.2:8080/api`, que es la dirección del localhost del ordenador desde el emulador Android.

4.2. Manual de Usuario

4.2.1. Administrador — Aplicación de Escritorio

Acceso al sistema

Abrir la aplicación e introducir el email y la contraseña del administrador. Pulsar INICIAR SESIÓN. El botón Ver/Ocultar permite visualizar la contraseña. Si las credenciales son incorrectas, el sistema muestra un mensaje de error específico sin cerrar la ventana.

Dashboard

La pantalla principal muestra cuatro tarjetas con el estado actual del taller (total de órdenes, pendientes, en proceso y listas para entregar), la tabla de órdenes recientes, el mapa visual de plazas y el panel de notificaciones recientes. Las notificaciones no leídas tienen borde rojo y las ya leídas aparecen en gris.

Gestión de Mecánicos

Desde el menú lateral, acceder a 'Mecánicos'. Para dar de alta a un nuevo mecánico, pulsar '+ Nuevo Mecánico'. El formulario incluye campos de nombre, apellidos, email, contraseña (mínimo 6 caracteres), teléfono (formato español), especialidad, número de empleado (único), turno y años de experiencia. El sistema valida automáticamente el email y el teléfono. Para editar, pulsar el icono de lápiz; para eliminar, el icono de papelera con confirmación.

Gestión de Clientes

Similar a la gestión de mecánicos. Los clientes pueden registrarse ellos mismos desde la app móvil o el administrador puede darlos de alta. El campo NIF se valida con el algoritmo oficial de la letra española.

Gestión de Vehículos

Al crear un vehículo, se selecciona el cliente propietario de un desplegable. La matrícula debe seguir el formato español: cuatro dígitos seguidos de tres letras (ej: 1234ABC). El año debe estar entre 1900 y el año actual.

Gestión de Órdenes de Trabajo

Para crear una orden, seleccionar el vehículo y el mecánico de los desplegables y escribir la descripción del problema (mínimo 10 caracteres). Al crear la orden, el mecánico y el cliente reciben notificaciones automáticas. Para cambiar el estado, pulsar el icono de recarga; el

sistema advierte si se intenta seleccionar el mismo estado que tiene la orden actualmente. Para ver el detalle completo con todas las notas, pulsar el icono de ojo.

Gestión de Horarios

Al crear un horario, seleccionar el mecánico, el día de la semana y las horas de entrada y salida en formato HH:mm. El sistema valida que la hora de salida sea posterior a la de entrada. Al crear o eliminar un horario, el mecánico recibe una notificación automática.

Gestión de Ausencias

La vista de ausencias muestra todas las solicitudes de los mecánicos con su tipo, fechas, motivo y estado. Las solicitudes pendientes tienen los botones de aprobar (verde) y rechazar (rojo). Al tomar una decisión, el mecánico recibe una notificación automática.

4.2.2. Mecánico — Aplicación Móvil

Órdenes de Trabajo

La pestaña 'Órdenes' muestra todas las reparaciones asignadas al mecánico. Cada tarjeta muestra el vehículo, la descripción del problema, el badge de estado con su color correspondiente, la fecha de entrada y el precio estimado. Al pulsar sobre una tarjeta se accede al detalle completo, que incluye el historial de todos los cambios de estado y diagnóstico realizados. Desde el detalle, el botón 'Actualizar orden' abre un modal donde se puede cambiar el estado (pulsando sobre los botones de colores) y el diagnóstico.

Horarios

La pestaña 'Horarios' muestra los turnos de la semana asignados por el administrador, con el día (badge azul para días laborables y rojo para fin de semana) y las horas de entrada y salida.

Ausencias

La pestaña 'Ausencias' muestra el historial de solicitudes con su estado (PENDIENTE en naranja, APROBADA en verde, RECHAZADA en rojo). Para solicitar una nueva ausencia, pulsar el botón '+ Solicitar'. En el formulario, seleccionar el tipo de ausencia, introducir las fechas en formato DD/MM/AAAA y un motivo opcional. La solicitud llega automáticamente al administrador mediante notificación.

Notificaciones

La pestaña 'Notificaciones' muestra todos los avisos recibidos. El badge rojo sobre el icono de la campana indica el número de mensajes no leídos. Pulsar sobre una notificación la marca como leída. El botón 'Marcar todas como leídas' permite limpiar todas las notificaciones pendientes de una vez.

Perfil

La pestaña 'Perfil' muestra el nombre, email y rol del mecánico. Para cambiar la foto de perfil, pulsar sobre el avatar circular. Se solicitará permiso de acceso a la galería del dispositivo. Si ya tiene foto, aparece el enlace 'Eliminar foto' para volver al avatar por defecto.

4.2.3. Cliente — Aplicación Móvil

Registro

Si el cliente no tiene credenciales, puede registrarse desde la pantalla de login pulsando '¿No tienes cuenta? Regístrate'. Introducir nombre, apellidos, email, contraseña (mínimo 6 caracteres) y teléfono (opcional). Tras el registro, iniciar sesión con las credenciales creadas.

Mis Vehículos

La pestaña 'Mis Vehículos' muestra los vehículos registrados a nombre del cliente por el administrador.

Mis Órdenes

La pestaña 'Mis Órdenes' muestra el estado actual de cada reparación. Pulsar sobre una orden muestra el detalle completo: una caja con el estado descriptivo en el color correspondiente (ej. 'Listo para recoger' en verde), el vehículo, el diagnóstico del mecánico, el mecánico asignado con su especialidad, los precios estimado y final, las fechas y el historial completo de cambios.

Notificaciones y Perfil

Funcionan igual que en el rol de mecánico. El cliente recibe notificaciones cada vez que hay un cambio en alguna de sus órdenes.

4.3. Credenciales de Prueba

Nombre	Email	Contraseña	Rol	Datos adicionales
Admin Taller	admin@taller.com	admin123	ADMIN	Usuario único administrador del sistema
Carlos García López	carlos@taller.com	admin123	MECANICO	EMP001, turno MAÑANA, especialidad Motor
Pedro Martínez Ruiz	pedro@taller.com	admin123	MECANICO	EMP002, turno TARDE, especialidad Electricidad
Ana Fernández Gil	ana@taller.com	admin123	MECANICO	EMP003, turno MAÑANA, especialidad Carrocería
Luis Gómez Torres	luis@cliente.com	admin123	CLIENTE	2 vehículos registrados
María López Torres	maria@cliente.com	admin123	CLIENTE	1 vehículo registrado
José Romero Vega	jose@cliente.com	admin123	CLIENTE	1 vehículo registrado
Elena Jiménez Paz	elena@cliente.com	admin123	CLIENTE	1 vehículo registrado

5. CONCLUSIONES FINALES

5.1. Valoración del Proyecto

El desarrollo del Sistema de Gestión de Taller Mecánico ha supuesto la integración práctica de la práctica totalidad de los conocimientos adquiridos a lo largo del ciclo formativo de Desarrollo de Aplicaciones Multiplataforma. El proyecto ha abarcado disciplinas que van desde el diseño de bases de datos relacionales hasta el desarrollo de interfaces gráficas de escritorio, pasando por el desarrollo de APIs REST seguras, el desarrollo móvil multiplataforma y la implementación de sistemas de comunicación entre múltiples plataformas.

La decisión de desarrollar tres aplicaciones interconectadas en lugar de una única ha supuesto un reto considerable en cuanto a la arquitectura y la coordinación entre plataformas, pero ha enriquecido enormemente el resultado final, que refleja un sistema real y funcional que podría implantarse en un taller mecánico real con mínimas adaptaciones.

5.2. Objetivos Alcanzados

A continuación se presenta una revisión de los objetivos planteados inicialmente y su grado de consecución:

Objetivo	Estado	Observaciones
Aplicación de escritorio con CRUD completo	COMPLETADO	CRUD de mecánicos, clientes, vehículos, órdenes, horarios, plazas y ausencias
Aplicación móvil con roles diferenciados	COMPLETADO	Dos perfiles completamente funcionales: mecánico y cliente
Registro de clientes desde la app móvil	COMPLETADO	Con validaciones de email, contraseña y teléfono
API REST robusta con 12 entidades	COMPLETADO	Spring Boot con Spring Security, JWT y JPA/Hibernate
Sistema de notificaciones automáticas	COMPLETADO	10 tipos de eventos diferentes con notificaciones en tiempo real
Historial de cambios por orden	COMPLETADO	Estados y diagnósticos con fecha, usuario y observación
Validaciones lógicas en formularios	COMPLETADO	Email, teléfono, NIF, matrícula, horas y más en ambas apps
Foto de perfil desde la galería	COMPLETADO	Para mecánicos y clientes, con opción de eliminar
Diseño estético coherente entre plataformas	COMPLETADO	Tema oscuro con paleta de colores definida y consistente
Gestión centralizada de errores	COMPLETADO	GlobalExceptionHandler con mensajes descriptivos
Endpoint de setup para primer admin	COMPLETADO	Autobloqueante tras la primera ejecución
Scheduler de ausencias próximas	COMPLETADO	Aviso diario a las 8:00h al administrador
Chat en tiempo real	NO COMPLETADO	Definido como trabajo futuro por limitaciones de tiempo

Objetivo	Estado	Observaciones
Notificaciones push nativas (FCM)	PARCIAL	Notificaciones internas implementadas; push nativas como trabajo futuro
Despliegue en la nube	NO COMPLETADO	Definido como trabajo futuro; sistema funcional en entorno local

5.3. Aprendizajes Obtenidos

- La importancia del diseño previo de la base de datos como cimiento del sistema. Un diseño bien pensado desde el principio evita refactorizaciones costosas durante el desarrollo.
- La complejidad de la integración entre múltiples plataformas con un backend común. La gestión del estado de autenticación, el manejo de errores y la serialización de datos requieren especial atención.
- El valor del tipado estático de TypeScript en el desarrollo móvil. La detección de errores en tiempo de compilación ha reducido significativamente los bugs en tiempo de ejecución.
- La necesidad de gestionar correctamente los hilos en JavaFX. Todas las llamadas a la API deben realizarse en hilos secundarios y las actualizaciones de la interfaz mediante Platform.runLater.
- La importancia de los mensajes de error descriptivos. Un sistema que comunica claramente los errores al usuario mejora enormemente la experiencia de uso y reduce el tiempo de depuración.
- La utilidad de Spring Security y JWT para implementar un sistema de autenticación robusto con control de acceso por roles de forma declarativa.

5.4. Trabajo Futuro

Las funcionalidades no implementadas por limitaciones de tiempo se plantean como mejoras futuras del sistema:

- Chat en tiempo real entre mecánicos y clientes mediante Firebase Realtime Database o WebSockets con Spring Boot.
- Notificaciones push nativas mediante Firebase Cloud Messaging (FCM) para que los avisos lleguen al dispositivo móvil aunque la aplicación esté cerrada.
- Despliegue del sistema completo en la nube (AWS o Azure) con Docker y configuración de CI/CD para automatizar el proceso de despliegue.
- Tests unitarios y de integración con JUnit 5 y Mockito para garantizar la calidad del código del backend.
- Módulo de facturación con generación de facturas en PDF mediante JasperReports.
- Versión iOS de la aplicación móvil, posible sin grandes cambios gracias al desarrollo multiplataforma con React Native.
- Panel de estadísticas avanzadas para el administrador: ingresos por período, tiempo medio de reparación, carga de trabajo por mecánico, etc.
- Recuperación de contraseña mediante email para los usuarios registrados.

6. BIBLIOGRAFÍA

- Documentación oficial de Spring Boot: <https://spring.io/projects/spring-boot>
- Documentación oficial de Spring Security: <https://spring.io/projects/spring-security>
- Documentación oficial de Spring Data JPA: <https://spring.io/projects/spring-data-jpa>
- Documentación oficial de JWT: <https://github.com/jwt/jwt>
- Documentación oficial de React Native: <https://reactnative.dev/docs/getting-started>
- Documentación oficial de Expo: <https://docs.expo.dev/>
- Documentación oficial de React Navigation: <https://reactnavigation.org/docs/getting-started>
- Documentación oficial de JavaFX: <https://openjfx.io/>
- Documentación oficial de ControlsFX: <https://controlsfx.github.io/>
- Documentación oficial de OkHttp: <https://square.github.io/okhttp/>
- Documentación oficial de Jackson: <https://github.com/FasterXML/jackson>
- Documentación oficial de Axios: <https://axios-http.com/docs/intro>
- Documentación oficial de MySQL: <https://dev.mysql.com/doc/>
- Documentación oficial de Hibernate/JPA: <https://hibernate.org/orm/documentation/>
- Documentación oficial de expo-image-picker: <https://docs.expo.dev/versions/latest/sdk/imagepicker/>
- BCrypt password hashing: <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>
- JWT (JSON Web Tokens): <https://jwt.io/introduction>